

Lecture2a

September 7, 2025

1 CS5489 - Machine Learning

2 Lecture 2a - Bayes Classifier

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

1. Bayes Classification and Generative Models
2. Parameter Estimation
3. Bayesian Decision Rule

```
[1]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
```

4 Classification Examples

- Given an email, predict whether it is spam or not spam.
 - **Email 1:** > There was a guy at the gas station who told me that if I knew Mandarin and Python I could get a job with the FBI.
 - **Email 2:** > A home based business opportunity is knocking at your door. > Don't be rude and let this chance go by. > You can earn a great income and find your financial life transformed. > Learn more Here. > To Your Success. > Work From Home Finder Experts
- Classification Examples
 - Given the *petal length* and *sepal width*, predict the type of iris flower.

Features

Versicolor

Virginica

```
[2]: # load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris2.csv', delimiter=',', skiprows=1)

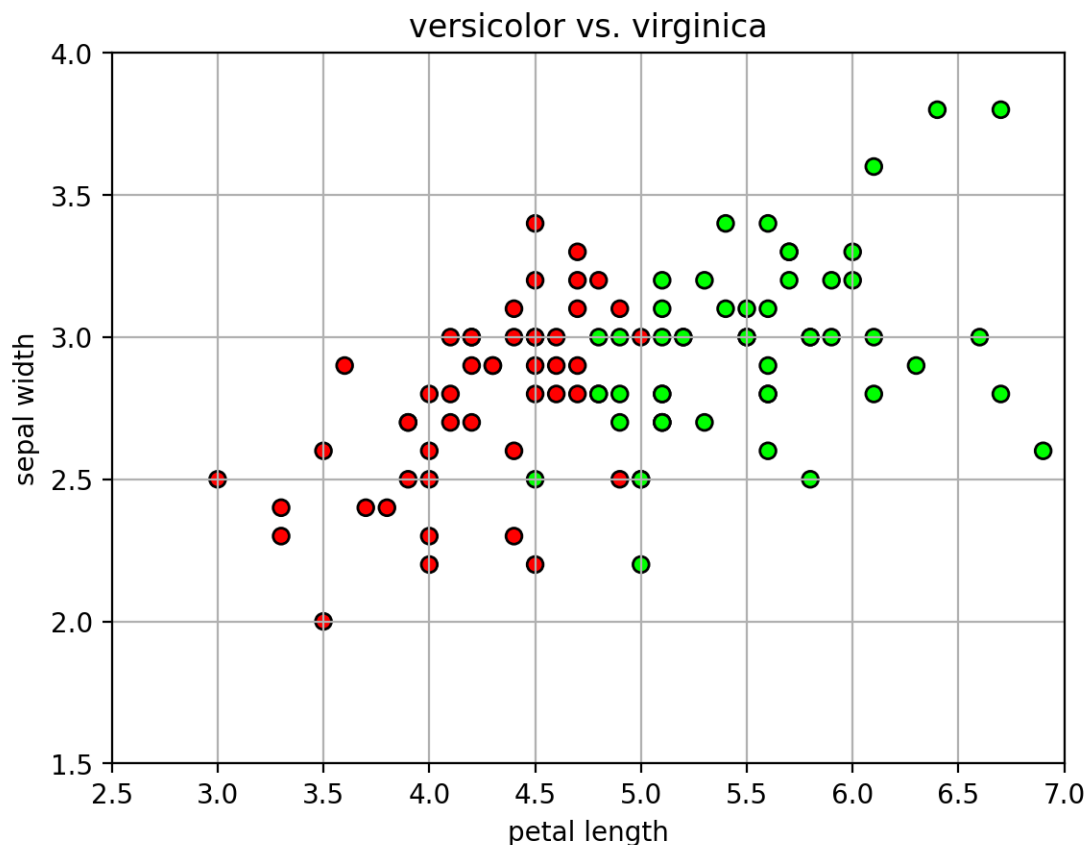
X = irisdata[:,0:2] # the first two columns are features (petal length, sepal
↪width)
y = irisdata[:,2]   # the third column is the class label (versicolor=1,
↪virginica=2)

# a colormap for making the scatter plot: class 1 will be red, class 2 will be
↪green
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',
↪["#FF0000", "#FFFFFF", "#00FF00"])

irisfig = plt.figure()
plt.scatter(X[:,0], X[:,1], c=y, cmap=mycmap, edgecolors='k')
# c is the color value, drawn from colormap mycmap
plt.xlabel('petal length'); plt.ylabel('sepal width')
plt.axis([2.5, 7, 1.5, 4]); plt.grid(True);
plt.title('versicolor vs. virginica');
plt.close()
```

```
[3]: irisfig
```

```
[3]:
```



5 General Classification Problem

- Observation $\mathbf{x}$ (i.e., features)
 - typically a real vector, $\mathbf{x} \in \mathbb{R}^d$.
 - **Example:** a 2-dim vector containing the petal length and sepal width.
 - * $\mathbf{x} = \begin{bmatrix} \text{petal length} \\ \text{sepal width} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$
- Class y
 - takes values from a set of possible class labels Y .
 - **Example:** $Y = \{\text{"versicolor"}, \text{"virginica"}\}$.
 - * or equivalently as numbers, $Y = \{1, 2\}$.
- **Goal:** given an observed features $\mathbf{x}$, predict its class y .

6 Probabilistic model

- To build a classifier we need to model the relationship between observations and classes.
- Model *how* the data is generated using probability distributions.
 - called a **generative model**.
 - build our assumptions about the world into the model.

- Generative model
 - 1) The world has objects of various classes.
 - 2) The observer measures features/observations from the objects.
 - 3) Each class of objects has a particular probability distribution of features.
- Need to define probability models for:
 1. the classes
 2. the features for each class

7 Class model

- Set of possible classes are Y
 - For example, $Y = \{\text{"versicolor"}, \text{"virginica"}\}$.
 - * or more generally, $Y = \{1, 2\}$.
- In the world, the frequency that class y occurs is given by the probability distribution $p(y)$.
 - $p(y)$ is called the **prior distribution**.
- **Example:** Bernoulli class distribution
 - $p(y = 1) = 0.4$
 - $p(y = 2) = 0.6$
 - “In the world of iris flowers, there are 40% that are Class 1 (versicolor) and 60% that are Class 2 (virginica)”
 - distribution: $p(y) = \pi^{\mathbb{1}(y=1)}(1 - \pi)^{\mathbb{1}(y=2)}$
 - * π is the parameter (e.g., 0.4)
 - * *Indicator function:*

$$\mathbb{1}(q) = \begin{cases} 1, & q \text{ is true} \\ 0, & \text{otherwise} \end{cases}$$

8 Learn from our data

- How to get the parameter $p(y = 1) = \pi$ for our model?
 - Assume we have collected some data, $D = \{y_1, \dots, y_N\}$.
- **Maximum Likelihood Estimation (MLE)**
 - find the parameter that maximizes the likelihood (log-likelihood) of observing the data.
 - $\pi^* = \operatorname{argmax}_{\pi} \sum_{i=1}^N \log p(y_i)$
 - * sum over the log-likelihoods of each sample (assumes samples are independent)
- if $y = 1$, then the log-likelihood is $\log(\pi)$, and if $y = 2$ the log-likelihood is $\log(1 - \pi)$.
- Sum over each sample:

$$\ell(\pi) = \sum_i \mathbb{1}(y_i = 1) \log \pi + \mathbb{1}(y_i = 2) \log(1 - \pi)$$

- Then, $\ell(\pi) = N_1 \log \pi + N_2 \log(1 - \pi)$
 - * where $N_1 = \sum_i \mathbb{1}(y_i = 1)$ = Number of 1's observed.
 - * and $N_2 = \sum_i \mathbb{1}(y_i = 2)$ = Number of 2's observed.
- Now solve for π by maximizing $\ell(\pi)$.
 - Take derivative and set to 0 to find the maximum.

$$\begin{aligned}\frac{d\ell(\pi)}{d\pi} &= \frac{N_1}{\pi} - \frac{N_2}{1-\pi} = 0 \\ N_1(1-\pi) - N_2\pi &= 0 \\ N_1 - (N_1 + N_2)\pi &= 0 \\ \Rightarrow \pi &= \frac{N_1}{N_1 + N_2}\end{aligned}$$

- $p(y = 1) = \frac{\text{number of examples of Class 1}}{\text{total number of examples}}$
- $p(y = 2) = \frac{\text{number of examples of Class 2}}{\text{total number of examples}}$

```
[4]: N1 = count_nonzero(y==1) # number of Class 1 examples
     N2 = count_nonzero(y==2) # number of Class 2 examples
     N  = len(y)              # total
     py = [double(N1)/N, double(N2)/N] # note: avoids integer division!
     print(py)
```

```
[np.float64(0.5), np.float64(0.5)]
```

9 Observation model

- We measure/observe a feature x
 - the value of the feature x *depends* on the class.
- The observation is drawn according to the distribution $p(x|y)$.
 - $p(x|y)$ is called the **class conditional distribution**
 - * “probability of observing a particular feature value x *given* the object is class y ”
 - Each class has its own class conditional:
 - * $p(x|y = 1)$ = distribution of features when its class 1
 - * $p(x|y = 2)$ = distribution of features when its class 2

9.1 Learn from the data

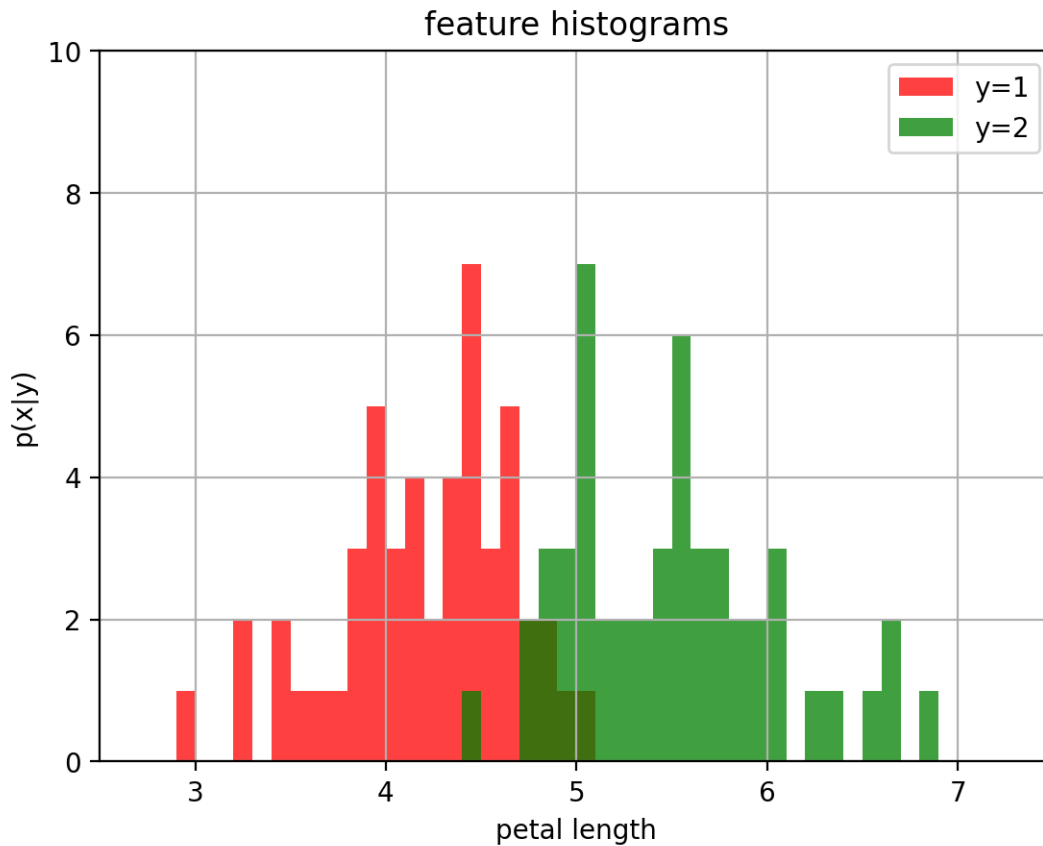
- Histograms for feature “petal length” for each class

```
[5]: # extract petal length for class 1 and class 2
     X1 = irisdata[y==1,0] # y==1 selects all rows where class=1
     X2 = irisdata[y==2,0]

     # plot a histogram for each
     ccdhist = plt.figure()
     binedges = arange(2.5, 7.5, 0.1)
     plt.hist(X1, binedges, facecolor='r', alpha=0.75, align='mid', label='y=1')
     plt.hist(X2, binedges, facecolor='g', alpha=0.75, align='mid', label='y=2')
     plt.legend()
     plt.xlabel('petal length'); plt.ylabel('p(x|y)')
     plt.title('feature histograms')
     plt.axis([2.5, 7.5, 0, 10]); plt.grid(True)
     plt.close()
```

```
[6]: ccdhist
```

```
[6]:
```



- **Problem:** looks a little bit noisy.
- **Solution:** assume a probability model for the class conditional $p(x|y)$

9.2 Gaussian distribution (normal distribution)

- Each class is modeled as a separate Gaussian distribution of the feature value
 - $p(x|y = c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{-\frac{1}{2\sigma_c^2}(x-\mu_c)^2}$
 - Each class has its own mean and variance parameters (μ_c, σ_c^2) .

```
[7]: # show mean/stddev example
tpdf = stats.norm(loc=2.3, scale=1.5)
gfig = plt.figure()
mymean = tpdf.mean()
mystd = tpdf.std()
tmpval = tpdf.pdf(mymean-mystd)
tmpval2 = tpdf.pdf(mymean-mystd*2)
xr = arange(-5,10,0.01)
xr2 = arange(mymean-mystd*2, mymean+mystd*2, 0.01)
```

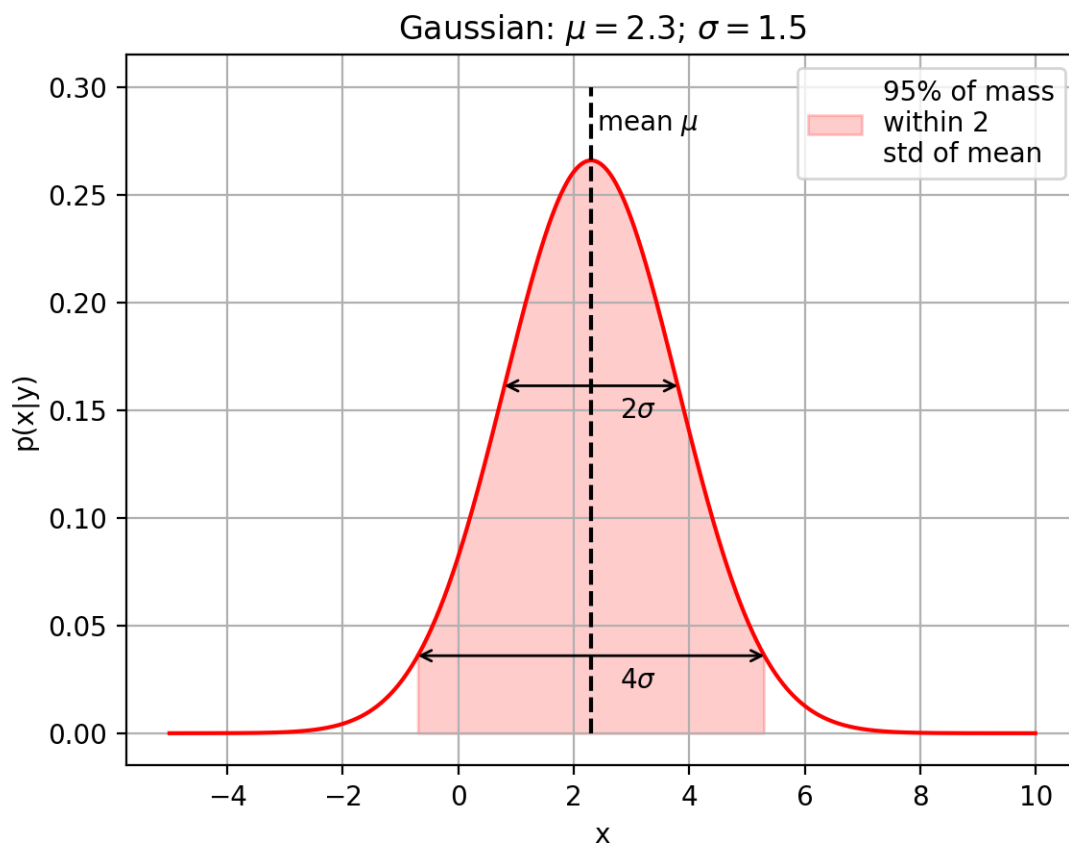
```

l1 = plt.fill_between(xr2, tpdf.pdf(xr2), alpha=0.2, color='r')
plt.plot(xr, tpdf.pdf(xr), 'r-')
plt.plot([mymean]*2, [0, 0.3], 'k--')
plt.text(mymean+0.1, 0.28, 'mean  $\mu$ ', horizontalalignment='left')
plt.annotate("", xy=(mymean-mystd, tmpval), xytext=(mymean+mystd, tmpval),
             arrowprops={'arrowstyle':'<->', 'shrinkA':0, 'shrinkB':0})
plt.text(mymean+0.5, tmpval-0.005, ' $2\sigma$ ', verticalalignment='top')
plt.annotate("", xy=(mymean-2*mystd, tmpval2), xytext=(mymean+2*mystd, tmpval2),
             arrowprops={'arrowstyle':'<->', 'shrinkA':0, 'shrinkB':0})
plt.text(mymean+0.5, tmpval2-0.005, ' $4\sigma$ ', verticalalignment='top')
plt.xlabel('x'); plt.ylabel('p(x|y)')
plt.title('Gaussian: ' + "$\mu=" + str(mymean) + "; $\sigma=" + str(mystd) + "\u2192$")
plt.grid(True)
plt.legend([l1], ['95% of mass\nwithin 2\nstd of mean'])
plt.close()

```

[8]: gfig

[8]:



10 MLE for Gaussian

- Set the parameters (μ, σ^2) to maximize the log-likelihood of the samples for that class.
 - Let $\{x_i\}_{i=1}^N$ be the observed features for class 1:

$$(\hat{\mu}, \hat{\sigma}^2) = \operatorname{argmax}_{\mu, \sigma^2} \sum_{i=1}^N \log p(x_i | y_i = 1)$$

- Then, the objective is $\ell(\mu) = \sum_{i=1}^N -\frac{1}{2\sigma^2}(x_i - \mu)^2 - \frac{1}{2} \log 2\pi\sigma^2$
- take derivative and set to 0

$$\sum_{i=1}^N \frac{1}{\sigma^2} (x_i - \mu) = 0$$

$$\sum_{i=1}^N x_i - N\mu = 0$$

$$\Rightarrow \mu = \frac{1}{N} \sum_i x_i$$

- Solution:
 - sample mean: $\hat{\mu} = \frac{1}{N} \sum_{i=1}^N x_i$
 - sample variance:

$$\hat{\sigma}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})^2$$

```
[9]: G1mean,G1std = stats.norm.fit(X1) # fit Gaussian to class=1 data
      G2mean,G2std = stats.norm.fit(X2) # fit for class=2 data
      print("mn={:0.5f} std={:0.5f}".format(G1mean, G1std))
      print("mn={:0.5f} std={:0.5f}".format(G2mean, G2std))
```

```
mn=4.26000 std=0.46519
mn=5.55200 std=0.54635
```

```
[10]: # create Gaussian class conditionals with the learned (mean,scale)
      pxy = [ stats.norm(loc=G1mean, scale=G1std), # class 1 (versicolor)
              stats.norm(loc=G2mean, scale=G2std) ] # class 2 (virginica)

      xr = arange(2.5,7,0.01)
      gcd = plt.figure()
      binedges = arange(2.5, 7.5, 0.1)
      plt.hist(X1, binedges, density=True, facecolor='r', alpha=0.20, align='mid')
      plt.hist(X2, binedges, density=True, facecolor='g', alpha=0.20, align='mid')

      lab1 = "y=1 ($\mu$={:.2f}, $\sigma$={:.2f})".format(G1mean, G1std)
      lab2 = "y=2 ($\mu$={:.2f}, $\sigma$={:.2f})".format(G2mean, G2std)

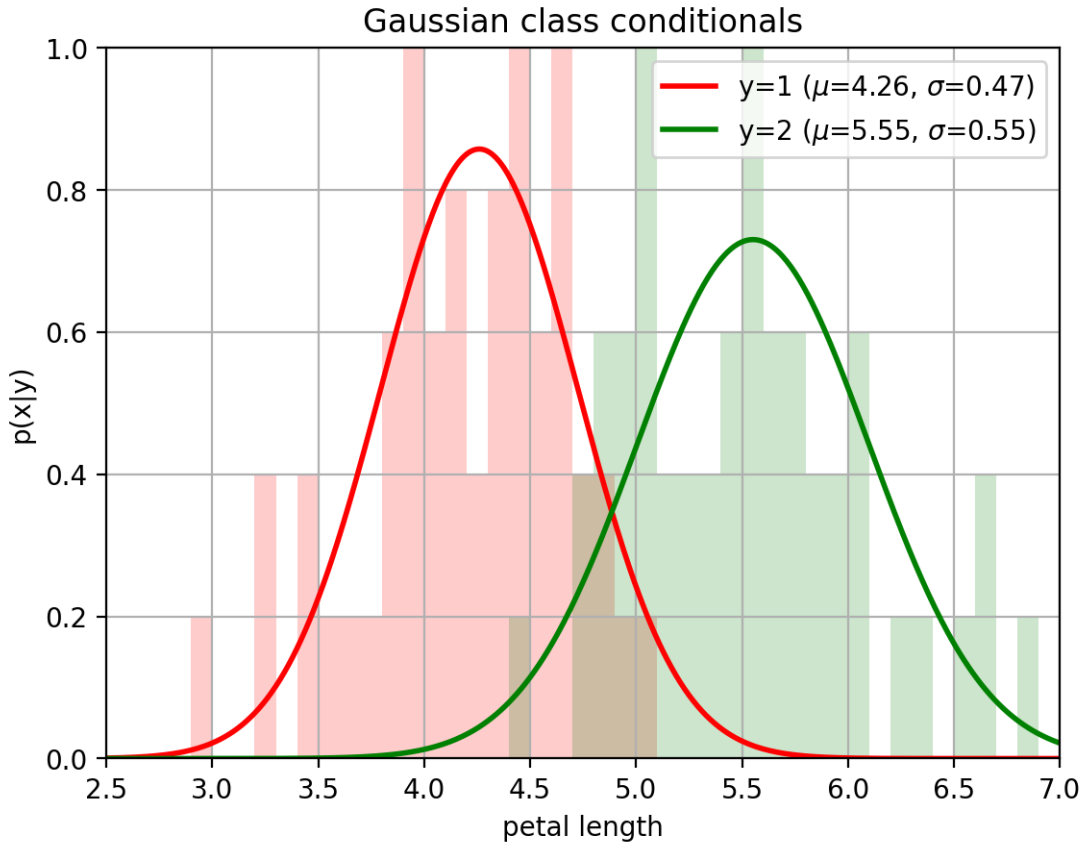
      plt.plot(xr, pxy[0].pdf(xr), 'r', lw=2, label=lab1)
      plt.plot(xr, pxy[1].pdf(xr), 'g', lw=2, label=lab2)
      plt.legend()
```



```
plt.ylabel('p(x|y)'); plt.xlabel("petal length")
plt.title('Gaussian class conditionals')
plt.axis([2.5, 7, 0, 1]); plt.grid(True)
plt.close()
```

```
[11]: gcd
```

```
[11]:
```



11 Bayesian Decision Rule

- The Bayesian decision rule (BDR) makes the optimal decisions on problems involving probability (uncertainty).
 - minimizes the *probability of making a prediction error*.
- **Bayes Classifier**
 - Given observation x , pick the class c with the *largest posterior probability*, $p(y = c|x)$.
 - * Probability of the class given observed x .
 - **Example:**
 - * if $p(y = 1|x) > p(y = 2|x)$, then choose Class 1
 - * if $p(y = 1|x) < p(y = 2|x)$, then choose Class 2
- Problem: we don't have $p(y|x)$!

- we only have $p(y)$ and $p(x|y)$.

12 Bayes' Rule

- The posterior probability can be calculated using Bayes' rule:

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

- The denominator is the probability of feature x , regardless of its class.
 * $p(x) = \sum_{y \in Y} p(x|y)p(y)$
- The denominator makes $p(y|x)$ sum to 1.
- Bayes' rule:

$$p(y = 1|x) = \frac{p(x|y = 1)p(y = 1)}{p(x|y = 1)p(y = 1) + p(x|y = 2)p(y = 2)}$$

- **Example:**

```
[12]: # calculate posteriors for each x
jxy = [pxy[0].pdf(xr)*py[0], # p(x/y=1)p(y=1)
        pxy[1].pdf(xr)*py[1]] # p(x/y=2)p(y=2)
px = jxy[0] + jxy[1] # p(x)

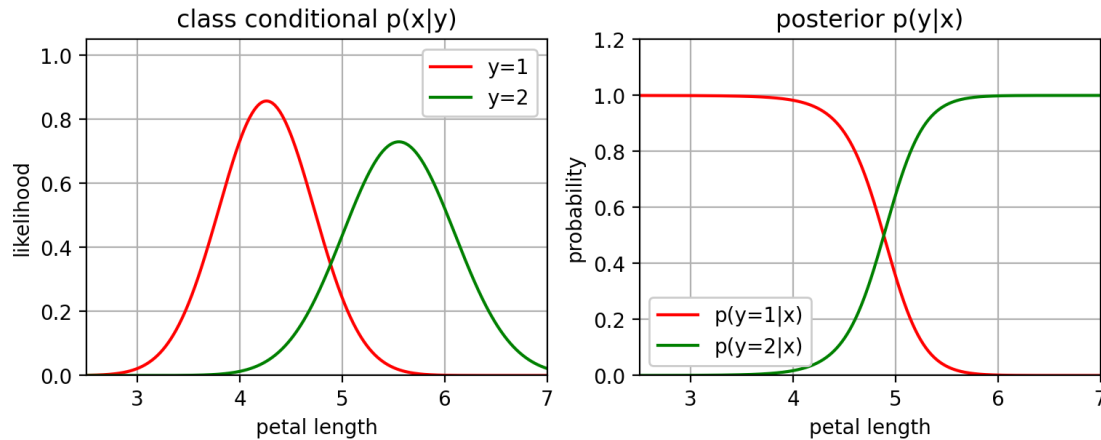
post = [ jxy[0] / px, # posterior for class 1
         jxy[1] / px ] # posterior for class 2

iris1dpost = plt.figure(figsize=(9,3))
plt.subplot(1,2,1)
plt.plot(xr, pxy[0].pdf(xr), 'r-', label="y=1")
plt.plot(xr, pxy[1].pdf(xr), 'g-', label="y=2")
plt.ylabel('likelihood'); plt.xlabel("petal length")
plt.title('class conditional p(x|y)')
plt.legend(framealpha=1, fontsize='medium')
plt.axis([2.5, 7, 0, 1.05]); plt.grid(True)

plt.subplot(1,2,2)
plt.plot(xr, post[0], 'r-', label="p(y=1|x)")
plt.plot(xr, post[1], 'g-', label="p(y=2|x)")
plt.ylabel('probability'); plt.xlabel("petal length")
plt.axis([2.5, 7, 0, 1.2]); plt.grid(True)
plt.title('posterior p(y|x)');
plt.legend(loc=0, framealpha=1, fontsize='medium')
plt.close()
```

```
[13]: iris1dpost
```

```
[13]:
```



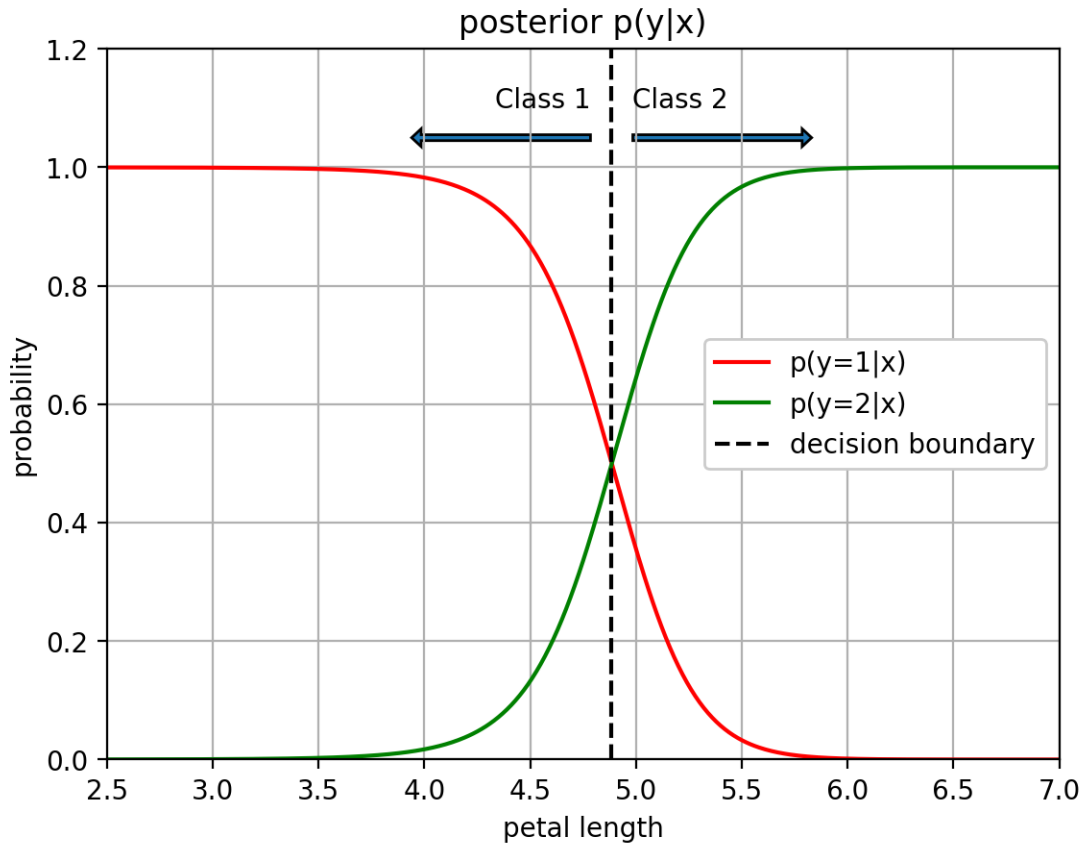
- The *decision boundary* is where the two posterior probabilities are equal
 - $p(y = 1|x) = p(y = 2|x)$

```
[14]: # find the index of the midpoint
midind = sum(post[0]>post[1])
midx   = 0.5*(xr[midind-1]+xr[midind])

iris1dpost2 = plt.figure()
plt.plot(xr, post[0], 'r-', label="p(y=1|x)")
plt.plot(xr, post[1], 'g-', label="p(y=2|x)")
plt.plot([midx, midx], [0,1.2], 'k--', label="decision boundary")
plt.ylabel('probability'); plt.xlabel("petal length")
plt.axis([2.5, 7, 0, 1.2]); plt.grid(True)
plt.title('posterior p(y|x)');
plt.arrow(midx-0.1,1.05,-0.8,0,width=0.01)
plt.arrow(midx+0.1,1.05,0.8,0,width=0.01)
plt.text(midx+0.1,1.10,"Class 2")
plt.text(midx-0.1,1.10,"Class 1", horizontalalignment='right')
plt.legend(loc=0, framealpha=1, fontsize='medium')
plt.close()
```

```
[15]: iris1dpost2
```

```
[15]:
```



13 Bayes rule revisited

- Bayes' rule: $p(y|x) = \frac{p(x|y)p(y)}{p(x)}$
- Note that the denominator is the same for each class y .
 - hence, we can compare just the numerators $p(x|y)p(y)$.
 - This also called the *joint likelihood* of the observation and class
 - * $p(x, y) = p(x|y)p(y)$
- **Example:**
 - BDR using joint likelihoods:
 - * if $p(x|y=1)p(y=1) > p(x|y=2)p(y=2)$, then choose Class 1
 - * otherwise, choose Class 2

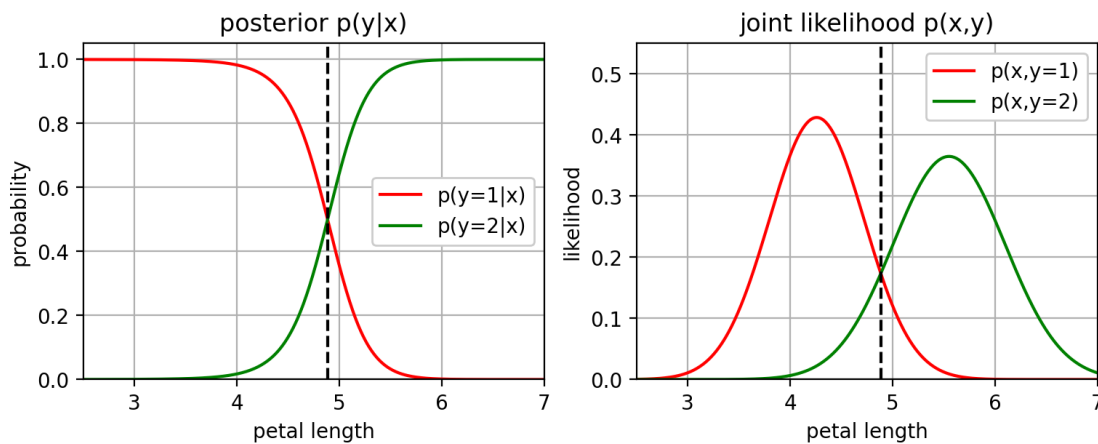
```
[16]: # calculate joint likelihood for each x
iris1djoint = plt.figure(figsize=(9,3))

plt.subplot(1,2,1)
plt.plot(xr, post[0], 'r-', label="p(y=1|x)")
plt.plot(xr, post[1], 'g-', label="p(y=2|x)")
a3, = plt.plot([midx, midx], [0,1.2], 'k--')
```

```
plt.ylabel('probability'); plt.xlabel("petal length")
plt.axis([2.5, 7, 0, 1.05]); plt.grid(True)
plt.title('posterior p(y|x)');
plt.legend(loc=0, framealpha=1, fontsize='medium')
plt.subplot(1,2,2)
plt.plot(xr, jxy[0], 'r-', label="p(x,y=1)")
plt.plot(xr, jxy[1], 'g-', label="p(x,y=2)")
plt.plot([midx, midx], [0,1.2], 'k--')
plt.ylabel('likelihood'); plt.xlabel("petal length")
plt.title('joint likelihood p(x,y)')
plt.legend(loc=0, framealpha=1, fontsize='medium')
plt.axis([2.5, 7, 0, .55]); plt.grid(True)
plt.close()
```

```
[17]: iris1djoint
```

```
[17]:
```



- Can also apply a monotonic increasing function (like log) and do the comparison.
 - Using log likelihoods:

$$\log p(x|y = 1) + \log p(y = 1) > \log p(x|y = 2) + \log p(y = 2)$$

- This is more numerically stable when the likelihoods are small.

```
[18]: # show joint likelihood and LL plots
iris1dLL = plt.figure(figsize=(9,3))

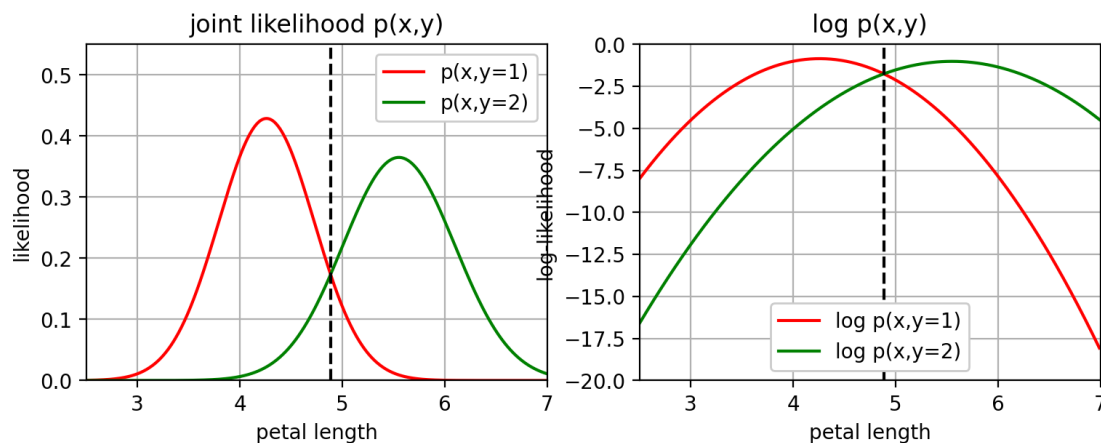
plt.subplot(1,2,1)
plt.plot(xr, jxy[0], 'r-', label="p(x,y=1)")
plt.plot(xr, jxy[1], 'g-', label="p(x,y=2)")
plt.plot([midx, midx], [0,1.2], 'k--')
plt.ylabel('likelihood'); plt.xlabel("petal length")
plt.title('joint likelihood p(x,y)')
plt.legend(loc=0, framealpha=1, fontsize='medium')
```

```
plt.axis([2.5, 7, 0, .55]); plt.grid(True)

plt.subplot(1,2,2)
plt.plot(xr, pxy[0].logpdf(xr) + log(py[0]), 'r-', label="log p(x,y=1)")
plt.plot(xr, pxy[1].logpdf(xr) + log(py[1]), 'g-', label="log p(x,y=2)")
plt.plot([midx, midx], [-20,5], 'k--')
plt.ylabel('log-likelihood'); plt.xlabel("petal length")
plt.title('log p(x,y)')
plt.axis([2.5, 7, -20, 0]); plt.grid(True)
plt.legend(loc=0, framealpha=1, fontsize='medium')
plt.close()
```

[19]: iris1dLL

[19]:



14 Bayes Classifier Summary

- **Training:**
 1. Collect training data from each class.
 2. For each class c , estimate the class conditional densities $p(x|y=c)$:
 1. select a form of the distribution (e.g. Gaussian).
 2. estimate its parameters with MLE.
 3. Estimate the class priors $p(y)$ using MLE.
- **Classification:**
 1. Given a new sample x^* , calculate the likelihood $p(x^*|y=c)$ for each class c .
 2. Pick the class c with largest posterior probability $p(y=c|x^*)$.
 - (equivalently, use $p(x^*|y=c)p(y=c)$ or $\log p(x^*|y=c) + \log p(y=c)$)

[]: